



MEMORIA PERSISTENTE PARA AGENTES DE IA

Musubi結び

Un sustrato de **memoria + orquestación** para agentes de IA — local-first, **model-free**, determinista, en un solo binario de Go. Lo que un proyecto aprende, no se pierde.

Versión

v0.83.1

Núcleo

Go · SQLite · MCP

Herramientas MCP

34

Fecha

2026·07·10

01	Resumen ejecutivo	10	Seguridad y multi-tenancy
02	Qué es Musubi	11	Escalabilidad y rendimiento
03	El problema que resuelve	12	Calidad de ingeniería
04	Arquitectura del sistema	13	Diferenciadores y foso
05	Los dos pilares	14	Viabilidad
06	Catálogo de 34 herramientas	15	Mercado y potencial
07	Eficiencia de tokens	16	Visión a largo plazo
08	Semántica model-free	17	Riesgos (honestos)
09	Cerebro híbrido y equipo	18	Conclusión + Apéndice

01 Resumen ejecutivo

TL ; DR

Musubi (結び, "nudo / atar" en japonés) es un servidor MCP (Model Context Protocol) escrito en Go que le da a un agente de IA una memoria persistente y eficiente en tokens, más un motor de orquestación multi-agente. Guarda lo que un proyecto aprende —decisiones, convenciones, bugs, hechos, gists de código— y en cada turno le inyecta al agente "solo lo relevante y solo lo nuevo". Todo corre local-first sobre SQLite, sin servicios externos obligatorios, y su núcleo es model-free: no hace inferencia ni gasta dinero — ranking, optimización de tokens y resolución de skills son deterministas y offline.

~40k

líneas de Go · 48% son tests

34

herramientas MCP en un solo binario

3

dependencias directas (uuid, yaml, sqlite)

~14MB

binario stripped, sin CGO

VEREDICTO DE MADUREZ

Auditoría profunda (v0.65.0, 5 agentes): ~8/10 — "producto de ingeniería maduro, production-ready para su nicho, feature-complete en los dos pilares". Identidad model-free / go-puro / local-first verificada en código.

EL FOSO (MOAT)

Único sustrato memoria + orquestación 100% model-free, determinista, Go-puro y local-first en un binario MCP. El foso = reproducibilidad + soberanía de datos + cero costo de inferencia + un ejecutable. Ningún competidor del SOTA es model-free.

Como sistema de autor, ya es sólido y usado a diario. La frontera activa —y el objeto de la última tanda de trabajo (Tracks 17–19, v0.83.1)— es convertirlo en un cerebro central de equipo 24/7: aislamiento multi-tenant sellado por contrato, identidad por-miembro, redacción forzada de secretos, backup off-host (DR) y observabilidad accionable. La trayectoria de re-auditorías fue 3.0 → 3.5 → 4.0 → 4.2 / 5 en semanas.

Antes que un producto, Musubi es una *postura*: la memoria y la coordinación de un agente no deberían costar inferencia, ni salir de tu máquina, ni depender de un tercero.

La definición en una línea

Un **servidor MCP en Go** que actúa como el **sistema nervioso de memoria y coordinación** de un agente de IA. Se conecta a Claude Code (o cualquier cliente MCP) y expone 34 herramientas para guardar y recuperar conocimiento, orquestar trabajo multi-paso y multi-agente, y auto-mantenerse.

"Musubi orquesta y persiste; el agente ejecuta."

— El contrato de diseño. El daemon nunca corre shell, red ni prompts por un paso de trabajo: es sustrato, no ejecutor. Esa línea divisoria es lo que lo mantiene determinista y auditable.

Tres principios que definen su identidad

LOCAL-FIRST La memoria vive en una base SQLite dentro de <code>.musubi/</code>. Ningún cambio requiere un servicio externo; las dependencias de red son opcionales y con fallback. Por default escucha solo en loopback.	MODEL-FREE Ranking, ahorro de tokens y resolución de skills son aritmética/SQL determinista, sin LLM en el camino crítico. No razona por su cuenta — su inteligencia es <i>combinatoria</i>, no generativa. Eso es una elección, no una carencia.	SOBERANO Y CHICO Un binario stripped (~14 MB), 3 dependencias directas, SQLite puro Go (sin CGo). Compila y corre en Windows, macOS y Linux, amd64 y arm64. Tus datos son tuyos.
--	---	--

El matiz honesto del "model-free"

El núcleo (ranking, tokens, skills, grafo) es model-free absoluto. La **búsqueda semántica** usa embeddings estáticos "*model-free at inference*": una tabla token→vector destilada offline (formato model2vec / POTION), sin forward pass en runtime — la misma categoría que servir vectores GloVe. No es "sin modelo en absoluto"; es "sin inferencia mientras corre". La única conexión de red a un proveedor de IA es `musubi calibrate` (opt-in, usa `count_tokens` — que *no* es inferencia y no factura — y vive **fuera** del daemon).

POR QUÉ EL NOMBRE

結び (musubi) es el arte japonés de anudar cordones — atar hilos sueltos en algo que sostiene. Es exactamente lo que hace: ata los aprendizajes dispersos de un proyecto (y de un equipo, y de las máquinas donde trabajás) en una memoria que persiste y te sigue.

■ Los agentes olvidan

Cada sesión de un agente arranca de cero. Las decisiones de arquitectura, los "por qué", los bugs ya resueltos, las convenciones del repo — todo se re-descubre o se re-explica. El conocimiento vive en la cabeza del humano y en el historial de chat, no en un sustrato consultable.

■ El contexto es caro

Volcar "todo lo que sabemos" en el prompt cada turno quema tokens (dinero + latencia + ventana de contexto). La mayoría de los sistemas de memoria RAG resuelven el *recall* pero no el *costo*: re-inyectan prosa completa una y otra vez.

■ El SOTA cobra inferencia y cede soberanía

Los sistemas serios de memoria (Mem0, Zep/Graphiti, Letta/MemGPT, Cognee) usan un LLM para extraer y organizar memoria, y la mayoría son servicios en la nube. Eso significa: costo por token en cada escritura/lectura, datos saliendo de tu infra, y comportamiento no determinista (la misma entrada puede guardarse distinto).

LA APUESTA DE MUSUBI

Resolver **recall Y costo Y soberanía Y determinismo** a la vez — aceptando a cambio un techo: sin un LLM de por medio, la *extracción* de prosa la hace el agente (que ya es un modelo), no el daemon. Musubi es el **sink** inteligente, no el extractor.

■ La forma de la solución

Musubi separa el **gist** (titular determinista de ~24 tokens) del **contenido completo**. Las búsquedas y el recall devuelven gists rankeados por relevancia que *caben en un presupuesto de tokens*; el contenido completo se **hidrata bajo demanda**, solo cuando hace falta. A eso suma inyección diferencial ("solo lo nuevo desde el último turno") y un **ledger** que contabiliza cada superficie de contexto contra un presupuesto de sesión. El resultado: memoria rica a costo de tokens mínimo.

Cuatro capas limpias, una sola base de datos, cero servicios obligatorios. El agente habla MCP; Musubi traduce a memoria determinista.

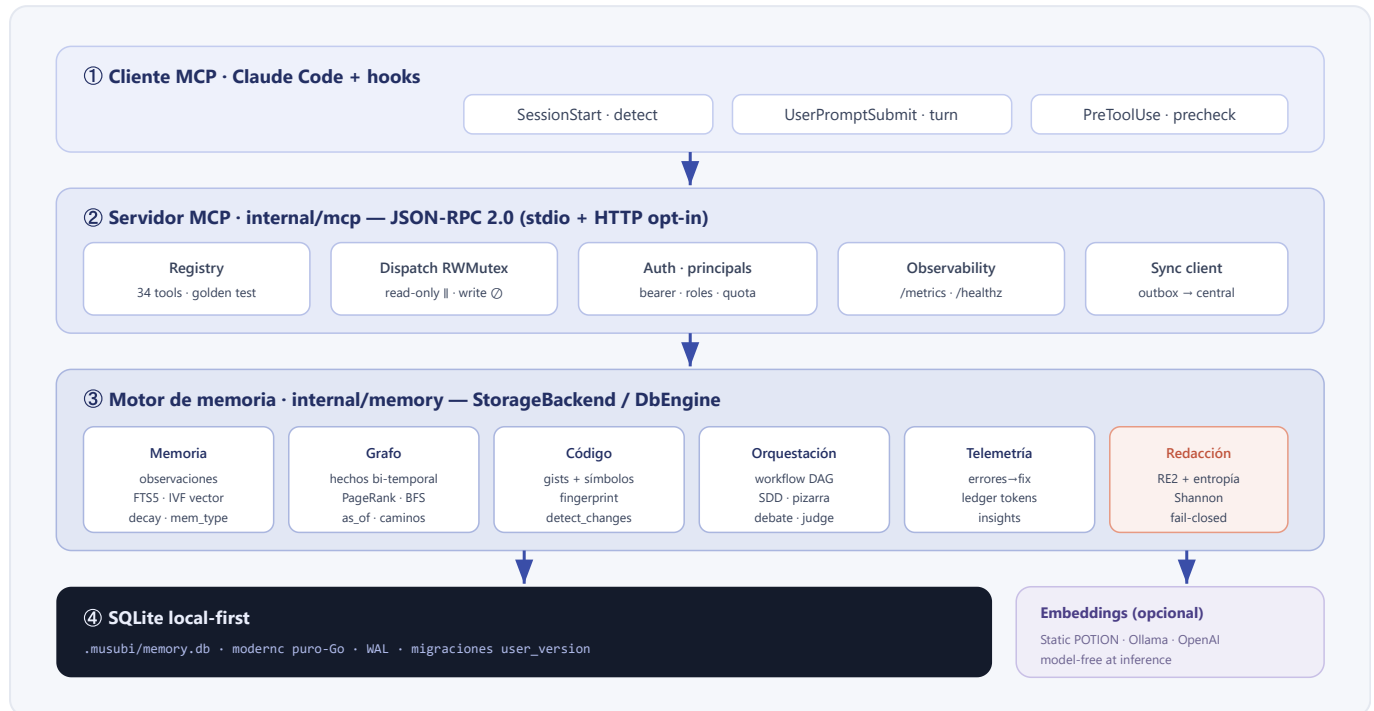


Figura 1 — Arquitectura de capas. Los hooks de Claude Code disparan al servidor MCP, que dispatchea contra el motor de memoria (lecturas concurrentes, escrituras serializadas) y persiste en una única SQLite local. Embeddings y sync son adyacentes y opcionales.

Stack

- ▶ Go 1.26+, sin frameworks. `CGO_ENABLED=0`.
- ▶ SQLite vía modernc.org/sqlite — traducción pura a Go, sin cgo (por eso cross-compila trivial y el binario "solo funciona").
- ▶ 3 deps directas: `uuid`, `yaml.v3`, `modernc/sqlite`.
- ▶ JSON-RPC 2.0 escrito a mano (evaluado vs SDK — ver `MCP_SDK_Evaluation.md`).

Modos de ejecución

- ▶ **Daemon long-running** (stdio por default; HTTP opt-in). Un proceso sirve las 34 tools.
- ▶ **Hooks one-shot**: `detect` (SessionStart), `turn` (UserPromptSubmit), `precheck` (PreToolUse) — inyectan contexto sin que el agente lo pida.
- ▶ **Modo servidor / cerebro central**: el mismo binario con `serve` sobre HTTP+bearer, detrás de una malla VPN (Tailscale/WireGuard). *Configuración, no motor nuevo.*

Decisión de trayecto (2026-06-30): elevar la **orquestración** a pilar paritario con la **memoria**. Musubi no es solo dónde vive el conocimiento — también es cómo se coordina el trabajo.



Una sola fuente de verdad (`registry.go · buildRegistry()`), con golden test que congela orden y conteo. Clasificadas read-only / write para dispatch concurrente seguro.

Memoria & recall

5

`save_observation` — guarda lección/decisión; indexa FTS5 + semántica; `mem_type` , `importance` .
`recall` — recall por presupuesto de tokens (relevancia+recencia+frecuencia+importancia).
`memory_expand` — hidrata contenido completo por id (perezoso).
`search_keyword` — full-text FTS5, siempre disponible.
`search_semantic` — similitud vectorial (requiere embeddings).

Grafo de conocimiento

3

`save_fact` — tripleta (s,p,o) bi-temporal; invalida por cardinalidad.
`recall_facts` — BFS o PageRank; `as_of` point-in-time; camino entre entidades.
`entity_context` — puente grafo↔prosa alrededor de una entidad.

Código

2

`save_code` — gist + símbolos con fingerprint de contenido.
`recall_code` — devuelve el gist + `fresh` según si el archivo cambió.

Tokens

1

`tokens` — ledger de sesión vs presupuesto blando (status/reset).

Cerebro híbrido / sync

3

`promote` — local→shared; redacta secretos al cruzar.
`sync_status` — salud del outbox (pending/sent/dead).
`sync_requeue` — reintenta dead-letter.

Skills

7

`detect_stack` · `search_skills` · `save_skill`
`author_skill` — gate scoreado 0–100 vs best-practices.
`resolve_skills` — activa según archivos + telemetría.
`log_skill_decision` · `discover_skills` (marketplace; solo descubre, nunca instala).

Orquestación / SDD

5

`workflow` — motor DAG resumible (start/next/complete/resume/rollback/verify...).
`sdd` — flujo Spec-Driven Development de 7 fases.
`phase` — pipeline explore→plan→code→verify.
`work` — pizarra: claim/lease/heartbeat/bidding/savings.
`debate` — debate multi-agente con quórum.

Conflictos de memoria

2

`conflicts` — relaciones semánticas pendientes de veredicto.
`judge` — related/compatible/scoped/conflicts/supersedes.

Observabilidad / mantenimiento

6

`maintain` — consolida duplicados + archiva frío.
`doctor` — diagnostica/repara la base (plan/dry-run/apply).
`insights` — resumen de observabilidad + hotspots.
`log_error` · `resolve_telemetry` — bucle de telemetría.
`detect_changes` — git diff + re-deriva símbolos; marca gists stale.

NOTA DE PRECISIÓN

El código real registra 34 herramientas (verificado en `registry.go`); el README todavía dice "27". Es un desfase de documentación a corregir — no de capacidades. Todos los nombres llevan el prefijo `musubi_`.

"No podés optimizar lo que no medís." Musubi es el raro sistema de memoria que trata los tokens como un presupuesto de primera clase, no como un efecto secundario.

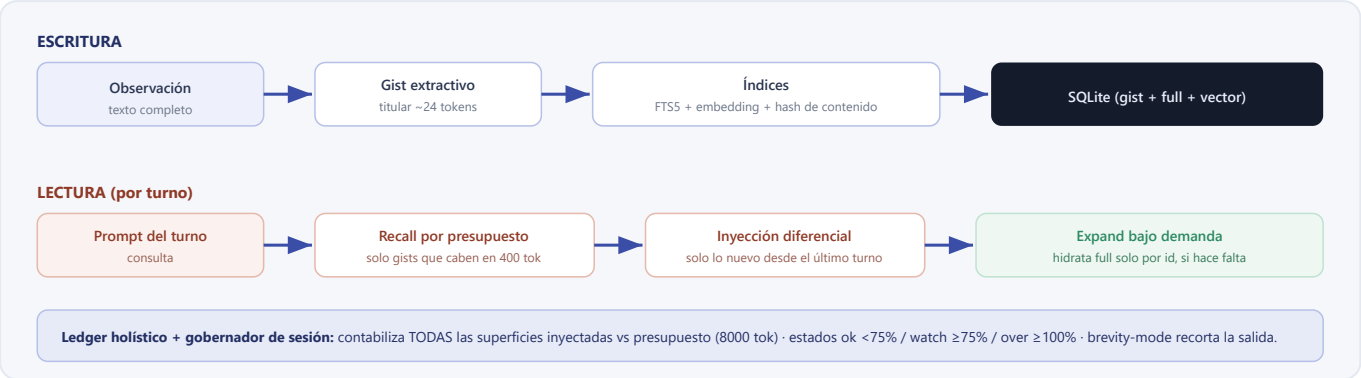


Figura 3 — Pipeline de eficiencia de tokens. El gist se separa del contenido; el recall empaqueta por presupuesto; la inyección es diferencial; el contenido completo se hidrata solo cuando se necesita. Un ledger mide todo el contexto contra un tope de sesión.

- ▶ **Gists extractivos deterministas** (`gist_max_tokens` = 24). Búsqueda y recall devuelven gists, no prosa.
- ▶ **Recall por presupuesto** — empaqueta solo lo que entra en `recall_token_budget` (400).
- ▶ **Hechos > prosa** — una triplete del grafo cuesta muchísimos menos tokens que recuperar prosa.
- ▶ **Memoria de código** — gist+símbolos evita re-leer archivos (el mayor costo de una sesión).
- ▶ **Ledger holístico** — cuenta cada superficie contra el presupuesto de sesión (8000).
- ▶ **Inyección diferencial** — por turno, solo lo nuevo/modificado.
- ▶ **Brevity mode** (opt-in: lite/full/ultra/auto) — recorta tokens de *salida*, preservando exacto código, comandos, rutas, versiones y flags.
- ▶ **Estimador por tipo** — divisores calibrados (prosa 4.0, código 3.4, JSON 2.6); afinables con `musubi calibrate`.

¿Cómo hacés búsqueda semántica sin un modelo corriendo? La respuesta es lo más ingenioso —y más incomprendido— de Musubi.

■ Embeddings estáticos (model2vec / POTION)

En vez de correr un sentence-transformer en cada consulta, Musubi usa una **tabla estática token→vector** destilada offline de un modelo real. En runtime solo hace **lookup + mean-pooling + L2-normalize** — sin forward pass, sin cgo, sin GPU. Es la misma categoría de "servir vectores GloVe": rápido, determinista, reproducible bit-a-bit.

El tokenizer es propio y bit-exacto vs HuggingFace (WordPiece BERT para inglés, Unigram/SentencePiece para multilingüe). La CI valida contra la tabla real `potion-multilingual-128M`.

■ Índice IVF que escala

Por debajo de un umbral de volumen, la búsqueda es full-scan exacto. Por encima, se entrena **solo** un índice vectorial **IVF** (inverted file) que hace la búsqueda sublineal — medido $\sim 3.7\times$ al pasar de 1k→10k embeddings (esperado $\sqrt{10}\approx 3.2\times$; una caída a full-scan daría $\sim 10\times$).

4 PROVIDERS, MISMA INTERFAZ

`none` (default, semántica apagada) · `static` (model-free at inference) · `ollama` (local) · `openai-compatible` (API, key desde env). Elegís tu punto en el espectro soberanía→potencia.

HONESTIDAD ESTRATÉGICA

La promesa semántica viene **apagada por default** (`Provider='none'`, tabla bring-your-own). Sin embeddings, "similar" cae a Jaccard de trigramas (sintáctico, no semántico). Encender la semántica *de verdad* —default `static` con harness de calidad de recall primero— es una de las piezas activas del roadmap. El andamiaje (IVF, tokenizers, recall-gate $R@10 \geq 0.80$) ya existe y está en CI.

La memoria no es "de la máquina" — es "del proyecto", y sigue al proyecto donde lo abras. Esta es la frontera donde Musubi pasa de herramienta personal a infraestructura de equipo.

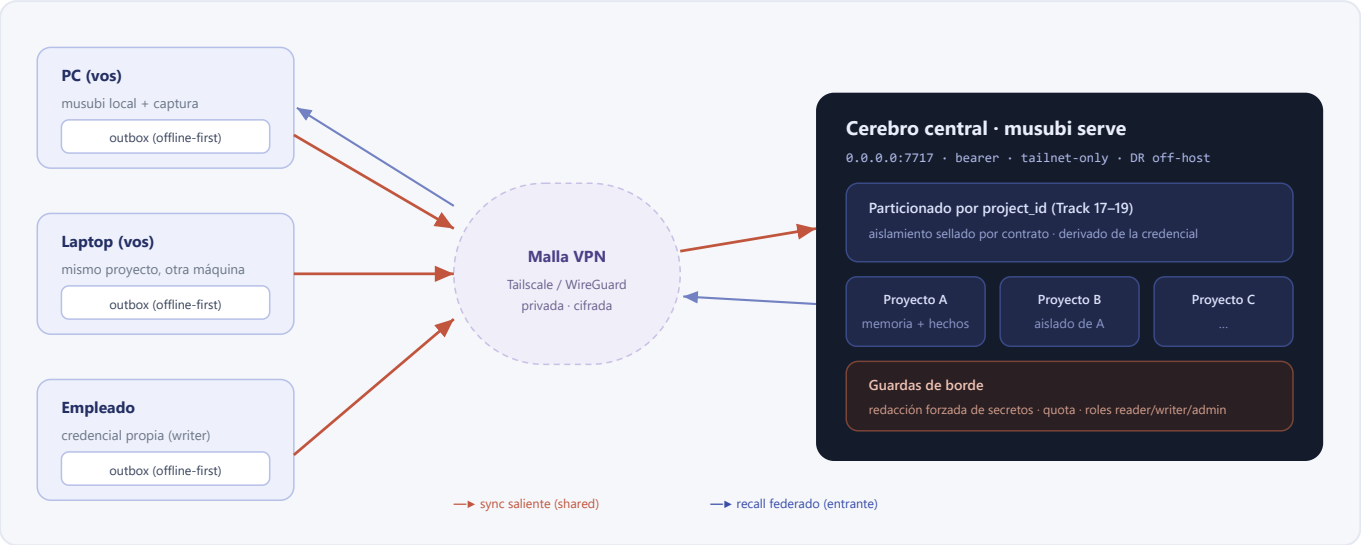


Figura 4 — Topología del cerebro híbrido. Cada máquina captura local (offline-first) y sincroniza lo `shared` por la malla VPN al cerebro central, particionado por `project_id`. Abrir el mismo proyecto en otra máquina recupera su memoria. El borde redacta secretos y aplica roles.

SCOPE LOCAL/SHARED

Toda observación nace `local` (privada). `promote` la marca `shared` y la encola en el outbox en la misma transacción (atómico, idempotente).

OUTBOX OFFLINE-FIRST

La captura no depende de la red: se bufferea y drena cuando hay conexión, con backoff. Rechazos por cuota son transitorios (reintenta), no dead-letter.

MULTI-TENANT SELLADO

El `project_id` se deriva de la **credencial**, nunca del cliente. Un writer del proyecto A no puede leer memoria del B — sellado *por contrato* (test de barrido).

■ Autenticación e identidad

- ▶ Bind no-loopback exige bearer token + TLS (o `allow_insecure_token` explícito tras un proxy). Token vacío ⇒ se niega a arrancar (nunca degrada a sin-auth).
- ▶ Registro de principals: token por miembro guardado como SHA-256 (nunca el crudo). Roles `reader` `writer` `admin`. Gestión con `musubi token new/list/revoke`; recarga en caliente.
- ▶ Anti fuerza bruta: logout por IP tras N fallos; comparación en tiempo constante.

■ Aislamiento multi-tenant

- ▶ `project_id` derivado de la credencial acota recall + lecturas de contenido. Escritura atribuida.
- ▶ Sellado por contrato (Track 19): test de barrido sobre todas las superficies de lectura + guarda que prohíbe una tool `readOnly` nueva sin clasificar.

■ Redacción de secretos

Guarda model-free, 0 dependencias (sin LLM ni red). Reglas RE2 por forma (AWS `AKIA`, GitHub `gh*_`, Stripe `sk_live`, Google `AIza`, JWT, PRIVATE KEY, `KEY=valor`) + catch-all de entropía de Shannon (≥ 4.5 bits/char) con allowlist. Forzada server-side, fail-closed, en todo ingest al pozo compartido.

LÍMITE HONESTO

La redacción es best-effort heurística: reduce, no garantiza. Está por debajo de gitleaks en cobertura; el plan es paridad con gitleaks-as-library + barrido periódico.

■ Transporte y DR

- ▶ TLS opcional + ACLs de Tailscale (defensa en profundidad); check anti DNS-rebinding en loopback; `MaxBytesReader` 4 MiB + timeouts anti slow-loris.
- ▶ Backup off-host consistente (`VACUUM INTO`) con fail-closed si no hay destino; alertas Prometheus (7 reglas) para staleness, outbox muerto, rechazos.

La escala se defiende en CI, no en una demo: guards deterministas que miden bytes-por-operación (no wall-time) y fallan el build si un cambio rompe la complejidad.

$\leq 20\times$

ratio B/op de Maintain
1k→10k (sub-cuadrático; era
~69× antes del fix)

$\sim 3.7\times$

IVF SearchVector 1k→10k
(sublineal; full-scan daría
~10×)

$< 20\text{MB}$

consumo basal de RAM
esperado

**$R@10$
 ≥ 0.80**

recall-gate en CI con tabla
POTION real

■ Lo que ya está probado

- ▶ 7 benchmarks (`bench_test.go`): save, recall, recall híbrido, FTS, vector, maintain, prime-context.
- ▶ **bench-guard** en CI corta el build si Maintain deja de ser sub-cuadrático o si el IVF cae a full-scan.
- ▶ SQLite en modo **WAL**; lecturas concurrentes, escrituras serializadas (RWMutex a nivel de dispatch por clasificación read-only/write de cada tool).

■ Fronteras conocidas de escala

- ▶ El IVF a **>10k** es opt-in (`MUSUBI_BENCH_SCALE`); benches a 100k/1M están en el roadmap (F3), aún no ejercitados.
- ▶ **Consolidate** materializa el corpus en RAM — a corpus muy grande necesita streaming.
- ▶ Señales de grafo (PPR) se recomputan por query sin caché (a diferencia del IVF, que sí cachea/throttlea).

Ninguna es reescritura: son optimizaciones puntuales con el camino ya identificado file:line en la auditoría de producibilidad.

1.26:1

ratio test : código (178 archivos `_test.go`)

70%

piso de cobertura con ratchet en CI

5

jobs de CI · 3 plataformas

0

TODOs reales · 0 panics en prod · vet limpio

■ CI (5 jobs, Go 1.26.5)

JOB	QUÉ GARANTIZA
test	vet + build + <code>-race</code> + coverage floor 70% + govulncheck
test-cross	build + test en Windows y macOS (sin cgo)
bench-guard	Maintain sub-cuadrático + IVF sublineal
recall-gate	R@10 ≥ 0.80 con tabla POTION real
lint	golangci-lint fijado + <code>.golangci.yml</code>

■ Prácticas que lo sostienen

- ▶ Golden test del registro: congela orden y conteo de `tools/list` — el catálogo no cambia por accidente.
- ▶ Fuzzing sobre parsers; tests de concurrencia/race; tests de aislamiento multi-tenant.
- ▶ Migraciones por `user_version`, cada una en su transacción con rollback y guarda de esquema fail-closed.
- ▶ 1 feature = 1 PR con bump semántico + CHANGELOG + tag → release cross-platform con checksums SHA-256 (12 assets).
- ▶ Auto-actualización con verificación de checksum (`musubi update`).

La pregunta no es "¿hace memoria?" — muchos la hacen. Es "¿lo hace sin cobrarte inferencia, sin sacarte los datos, y de forma reproducible?". Ahí Musubi está solo.

SISTEMA	MODEL-FREE EN INFERENCIA	LOCAL-FIRST SOBERANO	DETERMINISTA / REPRODUCIBLE	UN SOLO BINARIO	MEMORIA	ORQUES- TACIÓN
Musubi	Sí	Sí	Sí	Sí	Sí	Sí
Mem0	No (LLM)	Parcial	No	No	Sí	No
Zep / Graphiti	No (LLM)	Self-host	No	No	Sí (grafo)	No
Letta / MemGPT	No (LLM)	Self-host	No	No	Sí	Agente
LangGraph	No	Lib	Parcial	No	Estado	Sí
Temporal	Sí	Self-host	Sí	No (cluster)	No	Sí (durable)
Cursor / claude-mem	Agente	Local	No	No	Sí	No

Comparación conceptual por categoría de producto; los productos evolucionan. El punto no es "mejor en todo", sino que Musubi es el único que marca **Sí** en las cuatro columnas de identidad y cubre ambos pilares.

EL FOSO, EN UNA FRASE

Determinismo reproducible + soberanía de datos + cero costo de inferencia + un ejecutable. Nadie del SOTA de memoria (Mem0/Zep/Letta) es model-free; nadie del SOTA de orquestación (LangGraph/AutoGen/CrewAI/Temporal) unifica memoria rica en el mismo sustrato determinista.

EL LÍMITE INHERENTE (Y POR QUÉ ESTÁ BIEN)

Musubi **nunca comprenderá por sí solo**: no razona, no parafrasea, no infiere. Su inteligencia es *combinatoria*. Pero ese es justo el trato: el agente (que ya es un LLM) aporta el juicio; Musubi aporta la memoria barata, soberana y reproducible. División de trabajo, no carencia.

Dos auditorías multi-agente (5 y 72 agentes, ancladas en código real con verificación adversarial) dan la lectura más honesta: sólido para autor, endureciéndose para equipo, con la distancia medida y acotada.



Figura 5 — Trayectoria de madurez. Del veredicto baseline 3/5 (equipo/servicio 24/7) a 4.2/5 en tres tracks. Lo restante (encender semántica por default con harness; benches de escala 100k/1M) está acotado.

■ Viabilidad técnica: alta

- ▶ Cimientos de correctitud **nivel-4 reales**: outbox transaccional idempotente end-to-end, claim con lease auto-recuperado, migraciones en tx, transporte fail-closed, RRF de 6 señales.
- ▶ Ninguna de las brechas restantes requiere **reescritura** — son piezas acotadas con camino file:line.

■ Viabilidad de producto: en curso

- ▶ De "funcional-plus para un autor" a "cerebro de equipo 24/7": Tracks 17–19 ya cerraron DR, versión unificada, aislamiento por-credencial, redacción forzada, threat model y observabilidad accionable.
- ▶ Quedan F2 (semántica ON por default + paridad de embeddings en captura) y F3 (cuota + benches de escala).

■ Las 5 cosas que lo separaban de "producible" — y su estado

BRECHA (BASELINE)	ESTADO
DR del nodo central (SPOF)	cerrado backup off-host
Modelo de equipo (identidad, project_id)	cerrado T17–19
Semántica apagada por default	en curso F2
Operabilidad a escala (/metrics, cuota)	avanzado métricas+alertas
Borde de confianza (redacción)	forzado paridad gitleaks pendiente

■ Dónde gana claramente

- ▶ **Soberanía de datos obligatoria** — salud, legal, defensa, banca, on-prem regulado. La memoria del agente nunca sale de la infra; nada viaja a un tercero.
- ▶ **Equipos que no quieren pagar inferencia por memoria** — el costo marginal de guardar/recuperar es ~ 0 . A escala de miles de operaciones/día, eso es dinero real.
- ▶ **Reproducibilidad / auditoría** — misma entrada, misma salida. Crucial para pipelines regulados y para depurar comportamiento de agentes.
- ▶ **Edge / air-gapped** — un binario sin cgo que corre sin red ni GPU.
- ▶ **Ecosistema MCP** — al ser un servidor MCP, se enchufa a Claude Code, IDEs y cualquier cliente MCP sin integración a medida.

■ Potencial bruto

Musubi apunta a una **categoría** naciente: el *sustrato de memoria* + *coordinación model-free* para la era de los agentes. Si "cada equipo va a tener agentes" es cierto, cada equipo va a necesitar un lugar soberano, barato y determinista donde esos agentes recuerden y se coordinen.

LA TESIS DE ESCALA

Un cerebro central por organización, particionado por proyecto, alimentado por captura automática desde cada máquina de cada empleado — **la memoria institucional de la empresa, soberana y viva**. Toda la infra para esto ya está construida o en el roadmap inmediato.

■ Modelo de negocio plausible

- ▶ Core open-source (MIT) — adopción y confianza.
- ▶ Valor de pago en el **plano de equipo**: gestión de flota, dashboards, SSO/audit, soporte de despliegue del cerebro central.



Figura 6 — Horizonte. De cerebro de autor a memoria institucional soberana. Cada paso reusa el sustrato existente; ninguno pide cambiar la identidad model-free.

■ **Captura automática de equipo (la frontera activa)**

Que la memoria de un proyecto sea **central por naturaleza** y siga al proyecto entre máquinas y personas — con **atribución por persona** (saber si lo aportó Juan o Ana) y política "capturar todo, limpiar después" (dedup + decay). Reusa scope+outbox+redacción+project_id ya construidos; falta auto-escribir al central y auto-leer desde él.

■ **Provisioning y flota**

Un one-liner cross-platform que deja una máquina 100% lista: cerebro + skills + repos + devtools. La memoria institucional necesita que sumar una persona/máquina sea trivial y reproducible.

RIESGO	REALIDAD	MITIGACIÓN
SPOF del nodo central	Perder <code>memory.db</code> central = perder la memoria compartida.	Backup off-host consistente (VACUUM INTO) + alertas de staleness; DR probado.
Techo model-free	No razona/parafrasea/infiere por sí solo; la extracción de prosa la hace el agente.	Es una elección de identidad; el agente aporta el juicio. Señales estructuradas (commits, error→fix) sí son model-free.
Redacción best-effort	Heurística: reduce, no garantiza cero fugas de secretos.	Paridad con gitleaks-as-library + barrido periódico (TruffleHog) en el roadmap.
Semántica off por default	Sin embeddings, "similar" es sintáctico.	Andamiaje (IVF, tokenizers, recall-gate) listo; encenderla por default con harness es F2.
Escala no ejercitada >10k	Benches a 100k/1M pendientes; Consolidate en RAM.	Camino identificado (streaming, cuota LRU); guards de complejidad ya en CI.
Adopción / bus factor	Proyecto de autor; ecosistema MCP joven.	Core MIT, docs bilingües, CI robusta; la apuesta es que MCP se vuelva estándar.

Que esta tabla exista —y que cada fila tenga evidencia y plan— es en sí una señal de madurez: el proyecto se audita a sí mismo con agentes y verificación adversarial, y trata sus propios hallazgos como roadmap.

Musubi es una apuesta poco común: en un mercado que resuelve la memoria de agentes tirándole *más* inferencia y *más* nube, elige el camino contrario — **determinismo, soberanía y costo cero de inferencia, en un solo binario**. Esa disciplina no es un límite disfrazado: es un foso. Ningún competidor del SOTA marca las cuatro casillas de identidad a la vez, y casi ninguno unifica memoria rica con orquestación persistente en el mismo sustrato.

Técnicamente es **maduro** (~8/10, feature-complete en los dos pilares, CI con race/coverage/bench/recall gates). Como **producto de equipo** está a ~1 nivel de "producible 24/7", con la distancia medida, acotada y ya en ejecución (3.0 → 4.2/5 en tres tracks). Su **potencial bruto** es una categoría: el sustrato de memoria y coordinación soberano para la era de los agentes — la memoria institucional viva de una organización, sin peaje de inferencia.

EN UNA FRASE

Musubi convierte "trabajar" en "aprender", de forma barata, soberana y reproducible — y esa combinación no la ofrece nadie más.

■ Apéndice A — Métricas duras

Versión	v0.83.1	LOC	~40.000 (48% tests)
Tools MCP	34	Paquetes internal/	14
Deps directas	3 (uuid, yaml.v3, modernc/sqlite)	Binario	~14 MB stripped, sin cgo
Tests	178 archivos <code>_test.go</code>	Cobertura	≥70% (ratchet)
CI	5 jobs · Win/mac/Linux · Go 1.26.5	Recall	R@10 ≥ 0.80
Plataformas	amd64 + arm64 · Win/mac/Linux	Licencia	MIT

■ Apéndice B — Glosario

- ▶ **MCP** — Model Context Protocol; el estándar por el que un cliente (Claude Code) habla con servidores de herramientas.
 - ▶ **Gist** — titular extractivo determinista (~24 tokens) que representa una memoria en el recall.
 - ▶ **IVF** — inverted file index; estructura de búsqueda vectorial sublineal.
 - ▶ **RRF** — reciprocal rank fusion; fusiona múltiples señales de ranking.
- ▶ **Bi-temporal** — el grafo distingue "cuándo fue verdad" de "cuándo lo supimos"; permite consultas `as_of`.
 - ▶ **Outbox** — cola durable de sincronización saliente; garantiza entrega al central sin perder datos offline.
 - ▶ **Scope local/shared** — privado del proyecto vs. candidato al cerebro central.
 - ▶ **Model-free at inference** — sin forward pass de un modelo en runtime (tabla estática destilada).

■ Apéndice C — Fuentes

Este reporte se ancló en el código real del repositorio (`README.md`, `registry.go`, `scope.go`, `observability.go`, `static.go`, `redact.go`, `bench_test.go`, `.github/workflows/ci.yml`) y en dos auditorías multi-agente con verificación adversarial (v0.65.0, 5 agentes; producibilidad 2026-07-08, 72 agentes). Los datos de estado de tracks provienen de la memoria de proyecto de Musubi al 2026-07-10.